

Machine Learning Project Documentation

Deployment

1. Overview

In the deployment phase of this project, the trained machine learning model (Random Forest) was taken out of the laboratory environment and transformed into a real-world application where end-users can interact with it via the web. Throughout this process, the model and data preprocessing tools were serialized and integrated into a Python-based Streamlit web application (app.py). To ensure the application runs smoothly and consistently across different environments, all library dependencies were standardized in the requirements.txt file, making the system production-ready in a cloud environment.

2. Model Serialization

The trained machine learning model and data preprocessing steps were serialized and saved to disk in .pkl format using Python's built-in pickle library. To ensure system integrity and prevent data structure mismatches in the production environment, three different components were exported separately:

- **rf_model.pkl:** The trained Random Forest algorithm, along with its learned hyperparameters and tree structures.
- **scaler.pkl:** The scaling object fitted on the training data. This ensures that any new data entered into the application is standardized to the exact same scale as the training data before being fed into the model.
- **model_columns.pkl:** The names and exact order of the independent variables (features) expected by the model. This file acts as a critical control to prevent erroneous predictions that could occur if the column order changes in the production system.

3. Model Serving

The Streamlit framework was chosen to serve the model and execute the prediction process. The serialized files were integrated into app.py, allowing the model to be loaded into memory (RAM). The application is hosted on Streamlit Community Cloud (or a similar cloud platform) to ensure continuous accessibility, rapid deployment, and easy management. Instead of complex traditional server architectures, this platform—optimized for data science—allows the model to be served directly to the end-user through an interactive web interface.

4. API / Application Integration

Instead of establishing a traditional REST API endpoint, the model integration is handled directly through Streamlit's user interface components. Through input forms created within `app.py`, parameters received from the user are converted into a Pandas DataFrame in the background. This data table is structurally validated using `model_columns.pkl`, scaled using `scaler.pkl`, and then passed directly to the `rf_model.pkl` file for prediction. The result generated by the model is instantly displayed to the user both textually and visually via the Streamlit interface.

5. Security Considerations

To ensure system stability, input validation mechanisms were established at the frontend level. Using Streamlit's built-in interface components (such as sliders and constrained number inputs), users are prevented from entering incorrect data types or extreme outliers outside the defined boundaries. Additionally, because the application is served over the HTTPS protocol via Streamlit Cloud, the data transfer between the user's device and the server is encrypted, ensuring a secure connection.

6. Monitoring and Logging

The performance and operational health of the application are monitored using the server logs provided by the cloud platform where Streamlit is hosted. During runtime, any warnings, library conflicts, data conversion errors, or memory (RAM) overflow issues are recorded in the system logs. In the event of an application crash or slowdown, these logs can be reviewed via the developer dashboard to quickly perform debugging and version updates.